

ARTIFICIAL INTELLIGENCE

What is Artificial Intelligence

Artificial intelligence (AI) refers to the simulation of human intelligence in machines that are programmed to think, reason, and learn like humans. It involves developing computer systems and algorithms that can perform tasks that typically require human intelligence, such as visual perception, speech recognition, decision-making, problem-solving, and language translation. AI aims to create machines that can mimic and replicate human cognitive abilities.

There are two main types of AI: **Narrow AI** and **General AI**.

1. **Narrow AI (also known as weak AI):** Narrow AI is designed to perform a specific task or set of tasks. It focuses on solving specific problems and lacks the ability to generalize knowledge beyond its specific domain. Examples of narrow AI include voice assistants (such as Siri or Alexa), recommendation systems, image recognition systems, and chatbots.
2. **General AI (also known as strong AI):** General AI refers to machines that possess human-like intelligence and are capable of performing any intellectual task that a human being can do. These machines would have a broad understanding of various domains and possess the ability to learn and apply knowledge across different contexts. General AI is currently more of a theoretical concept and does not exist in reality.

Artificial intelligence encompasses several subfields, including machine learning, natural language processing, computer vision, robotics, expert systems, and neural networks. These subfields use different approaches and techniques to enable machines to acquire and process information, make decisions, and improve their performance over time.

Machine learning is a core component of AI and involves training algorithms to learn from data and make predictions or take actions based on that learning. It involves providing large amounts of data to algorithms and allowing them to automatically discover patterns and relationships within the data, enabling them to make accurate predictions or decisions.

Natural language processing (NLP) focuses on enabling machines to understand, interpret, and generate human language. It involves tasks such as speech recognition, language translation, sentiment analysis, and text summarization.

Computer vision deals with enabling machines to understand and interpret visual information. It involves tasks such as object recognition, image classification, and image generation.

AI has numerous applications across various industries, including healthcare, finance, transportation, entertainment, and manufacturing. It has the potential to revolutionize these industries by automating tasks, improving efficiency, enhancing decision-making, and enabling new capabilities.

However, AI also raises ethical and societal concerns, such as job displacement, privacy, bias in algorithms, and the potential for autonomous machines to make decisions that may have significant consequences. As AI continues to evolve, it is important to ensure its responsible development and deployment to maximize its benefits while minimizing potential risks.

FUNDAMENTAL PRINCIPLES OF ARTIFICIAL INTELLIGENCE

Here's a more detailed explanation of the fundamental principles underlying artificial intelligence :-

1. **Problem-solving** :- Problem-solving in AI involves developing algorithms and techniques to solve complex tasks or challenges efficiently and effectively. It encompasses various methods such as

planning, optimization, pattern recognition, and decision-making. AI systems are designed to analyze problem spaces, explore possible solutions, and select the best course of action to achieve desired goals or outcomes.

2. **Knowledge representation:** Knowledge representation in AI focuses on organizing and structuring information in a way that intelligent systems can utilize effectively. It involves creating models or frameworks to represent knowledge about the world, including facts, rules, concepts, relationships, and dependencies. These representations enable AI systems to reason, learn, and make informed decisions based on the available knowledge. Different techniques, such as semantic networks, ontologies, or knowledge graphs, are used to capture and represent information in a format that can be processed by AI algorithms.
3. Decision-making in AI involves selecting actions or choices based on specific criteria or objectives. It encompasses a range of techniques, including logical reasoning, probabilistic reasoning, and optimization. AI systems analyze available data, evaluate potential outcomes, and assess the likelihood of different scenarios to make informed decisions. They can rely on predefined rules, algorithms, or learned patterns to determine the most suitable course of action.

These three principles—problem-solving, knowledge representation, and decision-making—are fundamental to the field of AI. Problem-solving provides the framework for addressing complex tasks, while knowledge representation enables the organization and utilization of information. Decision-making allows AI systems to make intelligent choices based on analysis and evaluation of available data. By combining these principles, AI researchers and developers aim to create systems that exhibit human-like

intelligence, reasoning, and decision-making abilities.

Formal Definition of Artificial Intelligence

Artificial Intelligence, or AI for short, is a really clever technology that makes computers and machines act and think like humans. It helps them understand things, make decisions, and even do tasks that usually only humans can do. AI is used in many things we use every day, like voice assistants, video games, and even self-driving cars!

Now, let's define the four aspects of AI in simple terms:

1. **Thinking Humanly:** This means making computers and machines think and behave like humans do. It's like giving them a brain that helps them understand and respond to things just like we do. For example, if you ask a question, the computer should be able to understand it and give you a good answer, almost as if it were a person.
2. **Thinking Rationally:** This means making computers and machines think logically and make smart decisions. It's like teaching them to solve problems using rules and logic. For instance, if you give the computer some information, it should use that information to figure out the best solution to a problem, kind of like solving a puzzle.
3. **Acting Humanly:** This means making computers and machines behave like humans. It's like giving them the ability to do things that people can do, such as talking, listening, and even understanding emotions. So, if you tell the computer to do something, it should understand your instructions and follow them, just like a person would.
4. **Acting Rationally:** This means making computers and machines behave in a smart and logical way. It's like training them to make decisions that make sense

and lead to good outcomes. For example, if the computer is playing a game, it should make moves that are smart and help it win the game, kind of like a really good player.

WHAT IS AN AGENT

An agent refers to something that perceives its environment through sensors and takes actions in that environment through effectors. In the context of Artificial Intelligence, an agent is typically a computer program or system that is designed to interact with its environment in order to achieve certain goals or objectives.

Here are the key points to understand about an agent:

1. **Perception:** An agent perceives its environment using sensors. In the case of humans, our sensors include our eyes, ears, nose, skin, and other organs that enable us to gather information about the world around us. Similarly, in the case of a robot agent, it may have sensors like cameras, sound recorders, infrared range finders, or other specialized sensors that allow it to sense and gather data about its environment.
2. **Action:** An agent takes actions in its environment using effectors. As humans, our effectors include our hands, legs, mouth, and other organs that enable us to interact with the world. For a robot agent, effectors can be various types of motors, actuators, or other mechanisms that allow it to physically interact with objects or perform tasks in its environment.
3. **Environment:** The agent is situated within an environment. This environment can be real, physical surroundings, or it can be a virtual or simulated environment. The agent perceives the environment through its sensors and takes actions in the environment using its effectors.

4. **Goals and Objectives:** An agent typically has specific goals or objectives it aims to achieve. These goals could be defined by the designer of the agent or can be learned or adapted by the agent through machine learning or other techniques. The agent's actions and decisions are guided by its goals or objectives.

RATIONALITY VS OMNISCIENCE

Rational Agent: A rational agent is one that acts in a way that helps achieve its goals, based on its beliefs or understanding of the world. It does what it believes is the right thing to do given the information it has. However, a rational agent can make mistakes or encounter unforeseen events that affect its decisions.

Omniscient Agent: An omniscient agent knows everything, including the actual outcome of its actions. In reality, achieving omniscience (knowing everything) is impossible. So, while an omniscient agent can always make perfect decisions with 100% certainty, such agents do not exist in real life.

Expected Success: Rational agents make decisions based on what they expect to be successful, given their beliefs and understanding of the situation. They consider the likelihood of success based on available information. However, unexpected factors can sometimes lead to mistakes or failures.

Unpredictable Factors: Rational agents can make mistakes because there are often unpredictable factors or events that they couldn't foresee or perceive. In the example, crossing the street was a rational decision because, most of the time, it would have been successful. However, the falling banner was an unforeseen event that couldn't have been predicted.

To summarize, being rational means making decisions based on what is expected to be successful given the available information. Rational agents can still make mistakes due to unforeseen factors or events. An omniscient

agent, which knows everything and acts perfectly, is not possible in reality. So, we can't blame an agent for not considering or acting upon something it couldn't have known or perceived.

AGENT TYPES

- **Simple reflex agents:** These agents make decisions based solely on the current percept (input) they receive. They don't have memory or the ability to keep track of past actions or states. Simple reflex agents use condition-action rules to determine their actions. They follow a set of predefined rules that map specific percepts to specific actions without considering the overall context.
- **Model-based reflex agents:** These agents, in addition to the current percept, also maintain an internal model of the world. They use this model to keep track of the world state and update it based on percepts received. Model-based reflex agents can make more informed decisions by considering the current state and using their internal model to predict the effects of different actions.
- **Goal-based agents:** Goal-based agents have a goal or objective they are trying to achieve. They have access to information about the current state of the world and use it to determine the actions needed to reach their goal. These agents employ a planning process to generate a sequence of actions that lead to the desired outcome. They consider the current state, the desired goal state, and the available actions to choose the most appropriate action at each step.
- **Utility-based agents:** Utility-based agents make decisions based on the concept of utility or desirability. They assign a value or utility to different outcomes and select actions that maximize the expected utility. These agents consider not only the goal but also the relative desirability of different outcomes. Utility-based agents

can handle situations where multiple goals may conflict with each other by weighing the trade-offs between different options.

- **Learning agents:** Learning agents have the ability to improve their performance over time through experience. They learn from the feedback they receive from the environment and adapt their behavior accordingly. Learning agents can acquire knowledge and develop strategies to make better decisions in the future. They use techniques like machine learning and reinforcement learning to learn from their interactions with the environment and optimize their actions.

Each type of agent builds upon the previous one, increasing in generality and complexity. Simple reflex agents are the most basic, while learning agents are the most sophisticated and adaptable.

DIFFERENT CLASS OF ENVIRONMENTS

In designing successful agents, it is important to understand the type of environment with which the agent interacts. Environments are where agents operate and receive information. They can vary in terms of their characteristics, and these properties impact how agents perceive and act within them. Understanding these properties helps in designing effective strategies for different types of environments.

Fully Observable vs. Partially Observable: Fully observable environments provide agents with complete and accurate information about the current state. The agent can directly perceive all aspects of the environment that are relevant to its decision-making process. In contrast, partially observable environments do not provide complete information. The agent may have limited or incomplete knowledge about the environment, requiring it to infer or estimate the current state based on available observations.

Example :- Imagine a puzzle-solving game where the player can see the entire puzzle board at all times. The player has complete information about the positions of all the puzzle pieces, and they

can plan their moves accordingly. The environment in this case is fully observable because the player has access to all relevant information about the puzzle.

Example :- Consider a self-driving car navigating a busy city street. The car's sensors, such as cameras and lidar, provide it with information about its surroundings. However, due to factors like blind spots, occlusions, or limited sensor range, the car may not have complete visibility of the entire environment. The car must use the available sensor data to estimate the positions of other vehicles and pedestrians, even if it cannot directly perceive them. The environment in this case is partially observable because the car has limited or incomplete knowledge about the current state of the road.

Deterministic vs. Stochastic:

Deterministic: In a deterministic environment, the next state of the environment is completely determined by the current state and the action taken by the agent. This means that for a given current state and action, there is a unique mapping or predictable outcome that leads to the next state. The environment behaves consistently, and the agent can accurately anticipate the consequences of its actions.

Example: Let's say you have a ball and you throw it into the air. In a deterministic environment, the ball's motion would follow predictable laws of physics. If you know the initial position, velocity, and the force applied when throwing, you can precisely calculate the ball's trajectory and predict where it will be at any given time. The next state (position of the ball) is completely determined by the current state (initial position, velocity) and the action (throwing force).

Stochastic: In a stochastic environment, there is an element of uncertainty or randomness involved in the outcome. The next state may vary, even when the same action is taken from the same current state. The environment introduces some degree of unpredictability, and the agent needs to consider probabilities or uncertainties when making decisions.

Example: Let's consider a dice-rolling game. When you roll a fair six-sided die, the outcome (the number rolled) is uncertain and random. Even if you roll the same die from the same starting position multiple times, you may get different results each time. The next state (rolled number) is not completely determined by the current state (starting position) and the action (rolling the die). The randomness of the dice introduces variability and makes the environment stochastic.

Chess: Chess is considered a deterministic game. The rules of chess are well-defined, and the outcome of each move is determined solely by the current game state and the actions taken by the players. There is no randomness or element of chance involved in the game. Given the same initial game state and a specific sequence of moves, the game will always progress in the same manner. The deterministic nature of chess allows for the possibility of advanced analysis and strategy based on logical reasoning.

Self-driving Cars: Self-driving cars operate in a stochastic environment. While self-driving cars rely on advanced algorithms and sensors to make decisions, the driving environment itself is inherently unpredictable and subject to various uncertainties. Factors such as the behavior of other road users, changing weather conditions, or unexpected obstacles introduce elements of randomness and variability into the driving process. The same set of actions taken by a self-driving car in a particular situation may lead to slightly different outcomes due to these stochastic elements. As a result, self-driving cars need to adapt to the dynamic and uncertain nature of the driving environment.

Episodic vs. Sequential

Episodic environments divide the agent's interaction with the environment into separate episodes or individual tasks. Each episode is independent, and the agent's actions and outcomes do not affect future episodes. The agent starts anew with each episode. Non-episodic (sequential) environments have a continuous interaction, and the agent's actions can have long-term consequences that influence future

states and decisions. The agent's actions in the past can impact the future.

Chess (Episodic): Chess is typically played as a series of individual games or matches, with each game representing an episode. Each game begins with a specific starting position and ends when a player wins, loses, or the game results in a draw. The outcome of one game does not directly influence the subsequent games. Players start afresh with each game, and the previous game's results do not carry over or affect the future games. Therefore, chess fits the definition of an episodic environment.

Self-driving Cars (Non-episodic): In contrast, self-driving cars operate in a continuous and non-episodic environment. The driving experience is a continuous interaction rather than a sequence of separate episodes. A self-driving car continuously navigates and interacts with the road environment, responding to real-time changes in traffic, road conditions, and other dynamic factors. The actions taken by the car have long-term consequences, as the car's behavior can influence subsequent states and decisions. For example, the car's past actions, such as previous lane changes or braking maneuvers, may impact the current driving situation or influence future actions. Hence, self-driving cars are better categorized as operating in a non-episodic environment.

Static vs. Dynamic:

Static environments remain unchanged while the agent is making decisions. The state of the environment does not change unless the agent acts upon it. In contrast, dynamic environments undergo changes even without the agent's actions. The environment may evolve or be influenced by external factors, and the agent needs to adapt to the changing conditions.

Chess (Static): Chess can be considered a static environment. Once the game begins, the chessboard and the pieces remain unchanged unless a player makes a move. The positions of the chess pieces do not change autonomously during the game. The state of the chessboard

remains constant until a player makes a move. The environment does not evolve or undergo any changes unless acted upon by the players. Therefore, chess can be characterized as a static environment.

Self-Driving Cars (Dynamic): Self-driving cars operate in a dynamic environment. The road environment is subject to constant changes and fluctuations. Factors such as the movement of other vehicles, pedestrians, and objects, as well as changing traffic conditions and road infrastructure, make the driving environment dynamic. Self-driving cars need to continuously perceive and adapt to these dynamic changes to navigate safely and make appropriate driving decisions. The state of the environment evolves over time, influenced by external factors and the actions of other entities. Therefore, self-driving cars operate in a dynamic environment.

Discrete vs. Continuous:

Discrete environments have a limited number of distinct states or actions. The agent's actions and the environment's state can be clearly defined and categorized. In contrast, continuous environments have infinite or large sets of possible states or actions. The agent and the environment operate in a continuous space, requiring more complex and precise computations or techniques.

Chess (Discrete): Chess can be considered a discrete environment. The game state in chess is characterized by a finite set of distinct positions and configurations on the chessboard. Each chess piece has a defined set of possible moves, and the actions of the players are discrete and well-defined within the rules of the game. The state and actions can be clearly categorized into distinct categories. The discrete nature of chess allows for precise analysis and enumeration of possible moves and positions.

Self-Driving Cars (Continuous): Self-driving cars operate in a continuous environment. The driving environment is not limited to a finite set of distinct states or actions. Instead, it involves a continuous range of possibilities and variables. The car's perception of the environment, such as

the positions of other vehicles, pedestrians, or objects, as well as the car's own motion, is represented in continuous spaces. The car's actions, such as steering, acceleration, and braking, can be smoothly varied within continuous ranges. The continuous nature of the driving environment requires more complex computations and algorithms to process and make decisions based on the continuous data.

TECHNIQUES OF ARTIFICIAL INTELLIGENCE

let's delve deeper into the main techniques of AI, namely Machine Learning, Deep Learning, and Reinforcement Learning :-

1. Machine Learning (ML)

Machine Learning is a subfield of AI that provides systems the ability to learn and improve from experience without being explicitly programmed. It centers on the development of algorithms that can modify themselves to improve over time. There are three primary types of machine learning:

- **Supervised Learning:** In this method, the machine is taught using labeled data. For instance, if we're trying to create an algorithm that can identify cats in images, we'd train the algorithm using a large number of images that are labeled either "cat" (positive examples) or "not cat" (negative examples). The algorithm would then learn the characteristics that differentiate a cat from other entities.
- **Unsupervised Learning:** This is a type of machine learning where the algorithm is not provided with labeled data. The goal is to find hidden patterns or intrinsic structures from unlabeled data. Common unsupervised learning methods include clustering (grouping similar instances together) and dimensionality reduction (simplifying input data without losing too much information).
- **Semi-supervised Learning:** As the name suggests, this is a hybrid approach that uses both labeled and unlabeled data for

training. This is typically used when obtaining a fully labeled dataset is time-consuming or costly.

2. Deep Learning (DL)

Deep Learning is a subset of machine learning that is based on artificial neural networks with representation learning. Representation learning is the ability of a machine to automatically find the features needed for data classification. DL is especially good at identifying patterns in unstructured data, such as images, sound, and text.

The "deep" in deep learning denotes the depth of layers in a neural network. A typical deep learning model consists of many layers of artificial neurons, also known as nodes. These layers are interconnected in a way that mimics the structure of a human brain. Each node in a layer uses input from nodes in the previous layer, performs a computation, and passes the result to nodes in the next layer.

One crucial aspect of deep learning is its ability to automatically learn feature representations from raw data, eliminating the need for manual feature extraction. Applications of deep learning include image recognition, natural language processing, speech recognition, and more.

3. Reinforcement Learning (RL)

Reinforcement Learning is another subfield of machine learning where an agent learns to behave in an environment, by performing certain actions and observing the results/rewards of those actions. It is about taking suitable action to maximize reward in a particular situation. It is employed by various software and machines to find the best possible behavior or path it should take in a specific context.

Reinforcement learning differs from supervised learning in that it doesn't need labelled input/output pairs to be explicitly provided, and it doesn't need sub-optimal actions to be explicitly corrected. Instead, the focus is on exploring the environment, taking actions, and learning from the feedback.

A good example of reinforcement learning is a chess engine. Here, the agent decides upon a series of moves depending on the state of the board (the environment), and learns optimal strategies by playing numerous games and adjusting its policies based on the game results (win, lose, or draw).

In conclusion, all these techniques constitute the bedrock of AI. Machine Learning provides the foundational principles and methods that allow systems to learn from data. Deep Learning takes these concepts further by enabling the construction of neural networks that can process complex, unstructured data. Finally, Reinforcement Learning focuses on decision-making and the optimization of sequences of actions for an agent operating in an environment.

These fields continue to evolve, driven by advancements in computational power and the ever-increasing availability of data, and they are finding applications across a wide range of domains, including healthcare, finance, autonomous vehicles, and more.

Three different types of reasoning in AI

Here's a simpler explanation of the three types of reasoning in AI:

1. **Deductive Reasoning:** Deductive reasoning is when an AI system reaches a certain conclusion based on established facts or premises. It's like following a set of logical rules. For example, if we know that all cats have tails and Fluffy is a cat, we can deduce that Fluffy has a tail. Deductive reasoning guarantees that if the premises are true, the conclusion will also be true.

Deductive reasoning is like solving a puzzle using rules. Imagine you have a rule that says all birds have wings, and you know that a penguin is a bird. By using deductive reasoning, you can figure out that the penguin must have wings too because it fits the rule. So, deductive

reasoning helps us make sure that if the rule is true and the facts are true, then our conclusion will also be true.

2. **Inductive Reasoning :-** Inductive reasoning involves drawing conclusions based on patterns or trends observed in specific examples. It's like making educated guesses. For example, if we observe that every cat we've seen so far has been black, we might generalize and guess that all cats are black. Inductive reasoning suggests that the conclusion is likely, but not guaranteed to be true.

Inductive reasoning is like making guesses based on patterns you've noticed. For example, if you see your friend playing with three green toys, and then you see another friend playing with three green toys too, you might guess that all your friends like to play with three green toys. It's not always guaranteed to be true, but it gives you a good guess based on what you've seen.

3. **Abductive Reasoning:-** Abductive reasoning focuses on finding the best explanation for a set of observations. It's like solving a mystery by piecing together clues. For example, if we observe dark clouds, hear thunder, and see people carrying umbrellas, we might deduce that it's likely to rain. Abductive reasoning aims to find the most plausible explanation for the given evidence.

Abductive reasoning is like being a detective and finding clues to solve a mystery. Let's say you come home and find a broken glass on the floor, some water spilled, and your dog hiding under the table. You might think that the dog knocked over the glass and spilled the water because those clues fit together. Abductive reasoning helps us find the best explanation for what we see and understand what might have happened.

In simpler terms, deductive reasoning is about reaching guaranteed conclusions based on known facts, inductive reasoning involves making educated guesses based on patterns, and abductive reasoning is about finding the best explanation for a given set of observations. These types of reasoning help AI systems make decisions and draw conclusions based on available information.

KNOWLEDGE REPRESENTATION IN AI

Knowledge representation in AI is about how to store and organize information in an AI system. There are several ways to represent knowledge in AI :-

- **Semantic Networks:** These are graph structures used for representing knowledge in patterns of interconnected nodes and arcs. Nodes represent concepts and arcs represent relationships between those concepts. They're often used in natural language processing and in building expert systems.
- Imagine you have a map where different places are connected by lines. In a semantic network, instead of places, we have concepts or ideas, and the lines show how they are related. For example, you might have a concept for "dog" and another for "bark," and the line between them shows that dogs can bark. It helps the AI system understand how different ideas are connected.
- **Frames:** Frames are data-structures for representing a stereotyped situation. They are similar to object-oriented classes where you have a collection of properties and values describing a situation or object. For example, a "car" frame would include properties like color, model, manufacturer, etc.
- Frames are like containers that hold information about something. Think of it like a template with different slots to fill. For example, if we have a frame for a "car," it would have slots for color,

model, and manufacturer. We can fill in those slots with specific information like "red," "sedan," and "Toyota" to describe a particular car.

- **Logic:** Logic (like propositional logic or first-order logic) is a common way of representing knowledge in AI. Logical statements are used to represent facts about the world, and logical inference rules are used to reason about those facts.
- Logic is like a set of rules or statements that help the AI system understand facts and make deductions. It's like using puzzle pieces to figure out the answers. For example, if we know that "all birds have wings" and "penguins are birds," we can logically deduce that penguins must have wings too.
- **Probabilistic Models:** These models are used when the world is uncertain. Bayesian networks and Markov models are examples of probabilistic models where knowledge is represented in terms of probabilities.
- Probabilistic models are used when things are not certain or definite. It's like making educated guesses based on probabilities. For example, if you see dark clouds, you might guess that it's going to rain, but you're not 100% sure. Probabilistic models help AI systems make decisions based on how likely something is to happen.

Learning techniques in AI

Learning in AI refers to the ability of an AI system to improve its performance based on experience. Here are the major learning techniques in AI:

- **Supervised Learning:** In this technique, the AI system is trained on a labeled dataset. It's similar to learning with a teacher. The goal of the AI system is to learn a mapping from inputs to outputs.
- **Unsupervised Learning:** In this technique, the AI system is given unlabeled data and needs to find patterns or structure in that data. It's like learning without a teacher.
- **Reinforcement Learning:** This technique involves an agent interacting with its environment by taking actions, receiving rewards or penalties, and learning to make better decisions over time.
- **Deep Learning:** This is a technique where artificial neural networks with many layers ("deep" structures) learn from a large amount of data. Deep learning algorithms are used for complex tasks like image recognition, speech recognition, and natural language processing.

In summary, reasoning, knowledge representation, and learning techniques are core components of AI systems. They determine how an AI system thinks, how it stores and organizes knowledge, and how it learns from experience. Understanding these concepts is fundamental to understanding AI.

CHOOSING THE RIGHT KNOWLEDGE REPRESENTATION

Choosing the correct knowledge representation or learning technique refers to selecting the most suitable way to represent information in an AI system, or choosing the most appropriate learning method for a particular problem or task. The choice depends on the problem domain, the nature of the input data, and the desired output. Here are a few examples:

1. Knowledge Representation

Let's assume we're building a recommendation system for an e-commerce platform. We could represent knowledge using a **graph-based model**. Products can be nodes in the graph, and

relationships (like frequently bought together) can be edges connecting these nodes. This type of representation is conducive to making recommendations based on the connections in the graph.

In contrast, if we're developing a medical diagnosis system, we might choose a **rule-based representation**. Here, we could represent medical knowledge as a set of IF-THEN rules. For example, "IF the patient has a fever, cough, and loss of smell, THEN they may have COVID-19." This allows for logical reasoning to arrive at a diagnosis based on the symptoms presented.

2. Learning Techniques

Consider a scenario where you have a large set of labelled images, and you want to build an AI model to classify these images. In this case, **supervised learning** would be the most appropriate learning technique, and within that, **deep learning** with Convolutional Neural Networks (CNNs) would be a good choice given their excellent performance on image data.

On the other hand, if you're working with a large dataset of customer transactions, and you want to identify segments of similar customers, but you don't have any pre-existing labels, then **unsupervised learning** is a better choice. Techniques like clustering (e.g., K-means, Hierarchical Clustering) could be used to group customers based on their transaction behavior.

In a different context, suppose you are designing a system to control a self-driving car. The system needs to make a sequence of decisions (steer left, steer right, accelerate, brake, etc.), and the optimal decision depends on the current state of the environment (position of other cars, pedestrians, traffic lights, etc.). In this case, **reinforcement learning** would be an appropriate technique, as it is designed for learning optimal sequences of actions based on reward feedback.

Thus, the choice of knowledge representation and learning technique depends on the specific requirements of your problem and the nature of your data. Making the right choice is a critical step in developing effective AI solutions

The Role of AI in Gaining Insight into Intelligence and Perception

1. How AI Models Mimic or Enhance Human Intelligence and Perception

Artificial Intelligence (AI) models have increasingly been developed to imitate and enhance human cognitive abilities, which are the mental skills and processes that allow us to acquire and apply knowledge. These cognitive abilities include learning from experiences, understanding complex concepts, reasoning, problem-solving, decision-making, and adjusting to new situations.

AI systems can mimic human intelligence in numerous ways. One prominent method is Machine Learning (ML), a subset of AI that provides systems the ability to learn and improve from experience without being explicitly programmed. Much like how a child learns to differentiate between various animals, ML algorithms can be trained on large datasets to identify patterns and make decisions. For instance, a machine learning model can learn to differentiate between spam and non-spam emails based on a training dataset.

AI not only mimics human intelligence but can also enhance it. For instance, AI can process vast amounts of data much quicker than a human, identifying patterns and relationships that might be missed by human analysts. This ability can be used in numerous fields, from healthcare to finance, to improve decision-making and predictions.

2. The Role of AI in Cognitive Science and Neuroscience

Cognitive science is an interdisciplinary field that studies how information is represented and transformed in the brain. It involves research from psychology, linguistics, philosophy, and computer science. Neuroscience, on the other hand, is a discipline that studies the structure and function of the nervous system, focusing on the brain and its impact on behavior and cognitive functions.

AI plays a crucial role in both these fields. In cognitive science, AI models can be used to simulate cognitive processes, helping to test hypotheses about how the mind works. For instance, AI models can simulate how humans process language, providing insights into how we understand and generate speech.

In neuroscience, AI can help in understanding how the brain works at a deeper level. For instance, neural networks, a type of AI model, are inspired by the structure and function of the brain. These models have layers of interconnected nodes, or "neurons," that process and pass on information, similar to how neurons work in the brain. By studying these models, neuroscientists can gain insights into how complex brain networks function.

3. Analysis of AI Use Cases in Areas such as Natural Language Processing, Image Recognition, etc.

AI has numerous practical applications across various fields, including natural language processing (NLP) and image recognition.

NLP is a subfield of AI that focuses on the interaction between computers and humans through natural language. It enables computers to understand, interpret, and generate human language in a valuable way. Use cases of NLP include voice assistants like Siri and Alexa, machine translation tools like Google Translate, and sentiment analysis in social media monitoring.

Image recognition, another application of AI, involves teaching computers to recognize images. This technology is commonly used in a variety of fields, including healthcare, where it can help diagnose diseases; in autonomous vehicles, where it enables the vehicle to recognize obstacles; and in social media, for tagging and categorizing photos. AI models, particularly Convolutional Neural Networks (CNNs), are at the core of these image recognition tasks.

AI's ability to mimic and enhance human intelligence and perception, its role in cognitive science and neuroscience, and its wide range of

applications are significant aspects of the field. These topics provide a deeper understanding of how AI functions, its potential, and its impact across various sectors.

SEARCHING ALGORITHMS

In the field of Artificial Intelligence (AI), search strategies are algorithms or methods used to explore and traverse a problem space in order to find a solution. These strategies are employed when there is a need to navigate through a set of possible states, actions, or paths to reach a desired goal or solution.

Now, let's define informed and uninformed search strategies in detail :-

Uninformed Search: Uninformed search, also known as blind search, refers to search strategies that do not have any additional information about the problem other than the available actions and the current state. These strategies explore the problem space in a systematic manner without utilizing any heuristics or prior knowledge specific to the problem.

Uninformed search algorithms typically involve visiting and expanding nodes in a search tree or graph based solely on the available actions and the order in which they are encountered.

Examples of uninformed search strategies include

- **Breadth-First Search (BFS),**
- **Depth-First Search (DFS),**
- **Uniform Cost Search (UCS).**
- **Depth-limited search (DLS)**
- **Iterative deepening search (IDS)**

Uninformed search is generally more time-consuming and less efficient compared to informed search, especially for complex problems.

Informed Search: Informed search, also known as heuristic search, involves utilizing additional knowledge or heuristics to guide the search process towards more promising paths or solutions. Heuristics are rules or estimates based on domain-specific knowledge that can guide the

search algorithm to prioritize certain actions or paths over others.

Informed search algorithms make use of heuristic functions that estimate the desirability or quality of states or actions based on the available information. These functions provide a measure of how close a state or action is to the goal. Examples of informed search strategies include Best-First Search, Greedy Search, and A* (A-Star) Search.

Informed search strategies leverage heuristics to make more informed decisions and focus the search on paths that are likely to lead to the goal state more efficiently. However, the effectiveness of informed search heavily depends on the quality and accuracy of the heuristic function employed.

BREADTH FIRST SEARCH

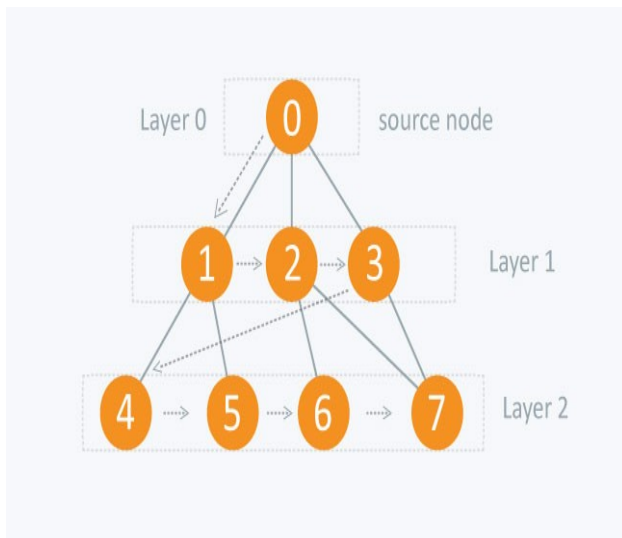
Breadth-First Search (BFS) is an uninformed search algorithm used to traverse or explore a graph or tree in a breadth-wise manner. It systematically explores all the nodes at the same depth level before moving on to nodes at the next depth level. BFS ensures that all nodes at a given depth are visited before moving to nodes at the next depth level.

Breadth-First Search guarantees that nodes at each depth level are visited before moving on to deeper levels. BFS explores the graph in a level-by-level manner, gradually moving away from the starting node.

Breadth-First Search is often used to find the shortest path or the minimum number of steps required to reach a goal state in an unweighted graph. It can also be used to explore or traverse a tree or graph systematically in a breadth-first fashion.

Note :- Breadth-First Search is an uninformed search algorithm, which means it does not consider any additional information or heuristics specific to the problem. It simply explores the graph in a systematic manner based on the

available connections and does not prioritize any particular path or node.



5. You continue this process, always selecting the path with the lowest cost, until you reach your destination (the goal node).

Uniform Cost Search ensures that you consider the cheapest paths at each step. It takes into account the cumulative cost from the start node to the current node and chooses the path with the smallest total cost.

This algorithm is particularly useful when you want to find the optimal path in terms of cost. It can be applied to various problems, such as finding the shortest route based on distance or finding the least expensive solution.

Remember, Uniform Cost Search focuses on cost rather than other factors like time or distance. It helps you make efficient decisions by prioritizing paths with lower cumulative costs at each step.

UNIFORM COST SEARCH (UCS)

Uniform Cost Search is an uninformed search algorithm that explores a graph or tree by considering the cost associated with each path. The goal of Uniform Cost Search is to find the path with the lowest total cost from the start node to the goal node.

Imagine you're in a maze where each path has a different cost. Uniform Cost Search helps you find the path that requires the least amount of effort (or cost) to reach your destination.

Here's a simpler explanation of how Uniform Cost Search works:

1. You start at a specific location (the start node) in the maze.
2. You explore all the paths connected to the start node and determine their costs.
3. You choose the path with the lowest cost and move to the connected node.
4. At the new node, you again explore all the paths connected to it and compare their costs.

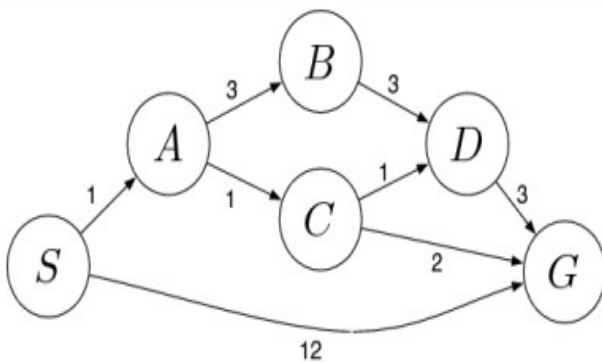
WHY ARE WE CALLING A UNIFORM COST SEARCH UNIFORM

We call it "Uniform" Cost Search because the algorithm assigns a uniform or equal priority to all paths based on their costs. It treats each path equally and explores them in a systematic manner without any additional biases or preferences. Unlike other search algorithms that may prioritize certain paths based on heuristics or estimations, Uniform Cost Search focuses solely on the cost of the paths.

Since Uniform cost search considers the cost , can't we say it is a heuristic approach ?

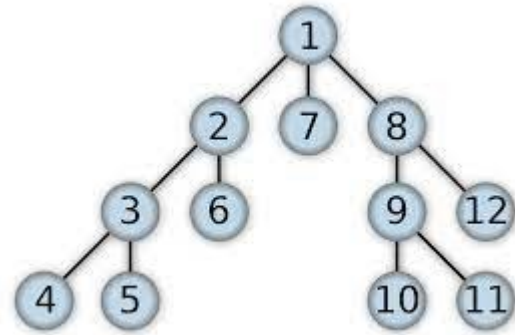
While Uniform Cost Search considers the cost associated with each path, it is still considered an uninformed search algorithm rather than a heuristic approach. In the context of search algorithms, heuristics typically refer to techniques that incorporate additional domain-specific knowledge or estimates to guide the search process. In Uniform Cost Search, the cost is considered as a known factor, but it does not involve any domain-specific knowledge or heuristics specific to the problem.

Uniform Cost Search does not rely on any prior knowledge or assumptions about the problem other than the given costs associated with each path. It explores the search space systematically by considering the accumulated costs, without any preconceived biases or estimations. The algorithm aims to find the path with the lowest total cost, but it does not utilize any heuristic functions or domain-specific insights.



- Repeat steps 2 and 3 until all nodes have been visited or the desired goal node is found.

Depth-First Search uses a stack (LIFO - Last-In-First-Out data structure) to keep track of nodes to visit. When traversing a branch, new nodes are pushed onto the stack, and when backtracking, nodes are popped off the stack.



DEPTH FIRST SEARCH

Depth-First Search (DFS) is an uninformed search algorithm used to traverse or explore a graph or tree. It follows a depth-wise exploration approach, meaning it explores as far as possible along each branch before backtracking.

Here's a simplified explanation of how Depth-First Search works:

- Start at a specific node in the graph or tree.
- Explore as deeply as possible along each branch before backtracking. This means visiting a neighboring node and continuing the exploration from there until reaching a dead end.
- Upon reaching a dead end, backtrack to the previous node and continue exploring other unvisited branches. This backtracking process allows for the exploration of alternative paths.

Breadth-First Search (BFS) Limitations:

- Memory Requirements:** BFS typically requires more memory compared to DFS. This is because BFS needs to store all the nodes at the current level in a queue, which can become memory-intensive if the branching factor is high or the search space is large.
- Time Complexity:** In the worst-case scenario, BFS can be slower than DFS. This is because BFS explores all nodes at each depth level before moving on to deeper levels, which may involve redundant exploration of nodes.

Depth-First Search (DFS) Limitations:

- Completeness:** DFS does not guarantee finding a solution if the search space contains cycles or infinite paths. If the solution exists deep within the search tree or graph and DFS explores an infinite path, it can get stuck and fail to find the solution.

2. **Lack of Optimal Solution:** DFS does not guarantee finding the optimal solution in terms of the shortest path. DFS may find a solution quickly, but it does not ensure that it is the best or shortest one.
- DFS is a search algorithm that can find a solution quickly, but it doesn't guarantee that the solution it finds is the best or shortest one.
- Imagine you're looking for a way to get from your home to a park. DFS might find a path that gets you to the park quickly, but it may not be the shortest or most efficient path. It could overlook a shorter route or take you on a longer detour.
- DFS focuses on exploring one path deeply before considering other paths, which can be helpful in certain scenarios. However, this depth-first approach doesn't always lead to the most optimal solution in terms of the shortest path or the most efficient way to reach your goal.

DEPTH LIMITED SEARCH

Depth-Limited Search is an algorithm that combines the advantages of breadth-first search (completeness) and depth-first search (space complexity) while addressing some of their limitations. It restricts the depth of exploration, ensuring that the search does not go beyond a certain depth level in the search tree or graph.

Here's a more detailed explanation:

1. **Advantage of Breadth-First Search (BFS):** BFS guarantees completeness, meaning it will find a solution if one exists. It explores all nodes at each depth level before moving on to deeper levels. However, BFS can have high space complexity because it needs to store all nodes at each level in memory.
2. **Advantage of Depth-First Search (DFS):** DFS is memory-efficient since it only

needs to store information about the current path. It can quickly find a solution by exploring deep paths first. However, DFS lacks completeness, as it can get stuck in infinite paths or cycles and may not find a solution even if one exists.

3. **Depth-Limited Search:** Depth-Limited Search addresses the limitations of both BFS and DFS. It sets a predefined depth limit, beyond which the search does not explore further. By restricting the depth, it prevents DFS from getting stuck in infinite paths while still being memory-efficient like DFS.
4. **Iterative Deepening Search:** An improved version of Depth-Limited Search is Iterative Deepening Search. It performs multiple depth-limited searches, gradually increasing the depth limit with each iteration. It starts with a shallow depth limit and gradually increases it until a solution is found. This approach combines the completeness of BFS (with increasing depth limits) and the space efficiency of DFS.

By using Depth-Limited Search or Iterative Deepening Search, we can explore the search space effectively, ensuring completeness while maintaining memory efficiency. These strategies are especially useful when the depth of the search tree or graph is unknown or large.

- Depth-Limited Search is an algorithm that limits the depth of exploration during a search process. It is a variation of Depth-First Search (DFS) that prevents infinite path exploration and helps overcome some of the limitations of DFS.

Here's a more detailed explanation of Depth-Limited Search:

1. **Depth-First Search (DFS) Recap:** DFS explores a path as deeply as possible before backtracking. It traverses through the graph or tree, moving from one node to another until it reaches a leaf node or

a specified condition is met. DFS can get stuck in infinite paths or deep branches, which hinders its completeness.

2. **Depth-Limited Search Approach:** Depth-Limited Search introduces a predefined depth limit to restrict the depth of exploration. It allows DFS to proceed until a certain depth level and then forces it to backtrack and explore other paths. By limiting the depth, the algorithm avoids getting trapped in infinite paths and ensures termination.
3. **Exploration Process:** The Depth-Limited Search algorithm operates as follows:
 - a. Start at the initial node and set the depth limit (e.g., a maximum depth level or a specific depth threshold).
 - b. Perform DFS exploration until the depth limit is reached or a goal state is found.
 - c. If the depth limit is reached and a goal state is not found, backtrack to the previous node and explore other unvisited paths.
 - d. Repeat steps b and c until a solution is found or all paths within the depth limit have been explored.
4. **Completeness:** Depth-Limited Search is not complete in itself, as it can miss solutions if they lie beyond the depth limit. However, it can be combined with Iterative Deepening Search (IDS) to achieve completeness. IDS performs multiple Depth-Limited Searches with increasing depth limits, ensuring that the search explores all paths up to a certain depth.

Depth-Limited Search strikes a balance between the memory efficiency of DFS and the avoidance of infinite path exploration. It allows for efficient exploration within a limited depth range while still maintaining termination. The effectiveness of Depth-Limited Search depends on selecting an

appropriate depth limit based on the problem's characteristics and search space complexity.

Note that Depth-Limited Search sacrifices completeness to gain efficiency and termination guarantees within a limited depth range. It is most suitable when the solution is expected to exist within a specific depth level or when the search space is vast and infinite path avoidance is necessary.

ITERATIVE DEEPENING SEARCH

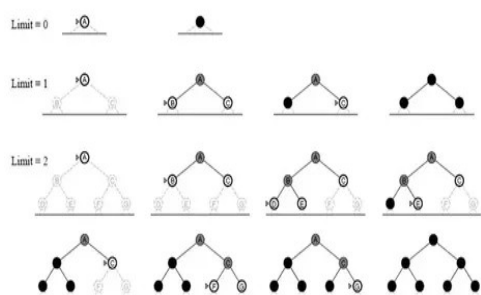
Iterative Deepening Search (IDS) is an algorithm that combines the benefits of depth-first search (DFS) and breadth-first search (BFS) by performing multiple depth-limited searches with increasing depth limits. It aims to achieve completeness while maintaining the efficiency of DFS.

Here's a more detailed explanation of Iterative Deepening Search:

1. **Depth-Limited Search:** IDS begins by performing a series of depth-limited searches. In each iteration, the algorithm applies depth-limited search with a specific depth limit, starting from 1 and incrementing the limit with each iteration.
2. **Exploration Process:**
 - a. Starting with a depth limit of 1, perform a depth-limited search from the initial node.
 - b. If the goal state is not found within the depth limit, increase the depth limit by 1 and repeat the depth-limited search.
 - c. Repeat steps a and b until the goal state is found or the entire search space is exhausted.
3. **Benefits of Iterative Deepening:** IDS overcomes the limitations of DFS and BFS in different ways:
 - a. **Completeness:** By incrementally increasing the depth limit in each iteration, IDS ensures that all nodes up to a certain depth level are explored. It

guarantees completeness, meaning that if a solution exists, IDS will eventually find it.

b. **Memory Efficiency:** IDS retains the memory efficiency of DFS since it only needs to store information about the current path being explored. It does not require the extensive memory usage of BFS, which stores all nodes at each depth level.



Iterative Deepening Search is a powerful search algorithm that combines the completeness of BFS and the memory efficiency of DFS. It is particularly useful when the search space is large and the depth of the optimal solution is unknown. IDS guarantees finding the optimal solution (in terms of path cost) if one exists while avoiding excessive memory usage.

INFORMED SEARCH

Informed Search, also known as heuristic search, is a search strategy that utilizes additional information or heuristics specific to the problem domain to guide the search process. It improves search efficiency by prioritizing the most promising choices for exploration.

Here's a breakdown of Informed Search:

1. **The Need for Informed Search:** In many search problems, having additional information about the problem domain can greatly enhance the search process. If we can order or prioritize the choices

based on their expected likelihood of leading to the goal state, the search becomes more efficient.

2. **Heuristics and Focused Search:** Informed search relies on domain knowledge or heuristics, which are rules, estimates, or insights that provide information about the problem domain. These heuristics help in determining which choices or paths are more likely to lead to the goal state, allowing for a focused search.
3. **Heuristic Information:** The additional information used in informed search is often referred to as heuristic information or heuristics. Heuristics provide an estimate of the cost or desirability associated with each state, guiding the search process. The heuristic information may not be entirely accurate but helps in making better decisions.
4. **Best-First Search:** Informed search is often implemented through a class of algorithms called Best-First Search. Best-First Search algorithms use heuristics to evaluate and prioritize the choices for exploration. They select the most promising choice or path based on the estimated desirability or cost and explore it further. **Examples of Best-First Search algorithms include Greedy Search and A* (A-Star) Search.**

Informed search allows the search algorithm to make informed decisions by leveraging heuristic information. By incorporating domain-specific knowledge, it directs the search towards more promising paths, potentially reducing the search effort and finding solutions more efficiently.

It's important to note that the effectiveness of informed search heavily depends on the quality and accuracy of the heuristic information used. Well-designed and accurate heuristics can significantly improve the search process, while poor or inaccurate heuristics may lead to suboptimal results.

HEURISTIC FUNCTION

Informed search is a strategy that helps an AI system search for a solution more efficiently by using information about the problem. It's like having a map or clues to guide you towards the right direction when you're searching for something. The AI system uses this information to make better decisions and explore the most promising options first.

Heuristic Information: Heuristic information is the type of information that the AI system uses to estimate the cost or value of different choices. It's like having hints or educated guesses about the best path to take. This information may not always be completely accurate, but it provides useful guidance to the AI system when making decisions.

Best-First Search: Best-first search is a specific type of informed search algorithm. It's like following a trail of breadcrumbs that leads to the most promising places. In best-first search, the AI system uses heuristic information to evaluate the available choices and selects the one that appears to be the most promising. It prioritizes exploring the options that are likely to lead to the goal state more quickly and efficiently.

To summarize, informed search is a strategy that helps the AI system make smarter decisions by using heuristic information. This information provides hints or estimates about the best choices to make. Best-first search is a specific type of informed search where the AI system selects the most promising option based on the heuristic information to reach the goal state more efficiently.

In best-first search algorithms, an important component is the heuristic function, denoted as $h(n)$. The heuristic function provides an estimated cost of the cheapest path from the current state at node n to a goal state. It utilizes domain-specific knowledge to guide the search algorithm.

Heuristic functions play a crucial role in informed search by imparting additional knowledge about the problem. They estimate the goodness of a

node based on the current state description and the domain-specific information available. By using the heuristic function, the search algorithm can make informed decisions and prioritize nodes that appear most promising in terms of reaching the goal state efficiently.

The evaluation function $f(n)$ is defined as the sum of $g(n)$ and $h(n)$, where $g(n)$ represents the cost of reaching a particular node from the starting state, and $h(n)$ represents the estimated cost or distance from that node to the goal state. The evaluation function $f(n)$ is used to compare and evaluate different nodes in the search tree.

Let's break down the components and explain the equation $f(n) = g(n) + h(n)$ in simpler terms:

- $g(n)$: In the context of a route-finding problem, $g(n)$ represents the cost of reaching a particular node (or state) from the starting node. It is like keeping track of the total distance or cost traveled so far in the search path.
- $h(n)$: In the context of a route-finding problem, $h(n)$ represents the estimated cost or distance from a particular node to the goal state. It is like having an estimation of how much more distance or cost is needed to reach the goal.
- $f(n)$: The evaluation function $f(n)$ is the sum of $g(n)$ and $h(n)$. It combines the cost incurred so far ($g(n)$) with the estimated remaining cost to reach the goal ($h(n)$). The evaluation function helps in comparing and selecting nodes in the search process.

To put it simply, in a route-finding problem, $g(n)$ represents the cost traveled so far, $h(n)$ estimates the remaining cost to reach the goal, and $f(n)$ is the sum of these costs. By evaluating $f(n)$, the algorithm can prioritize nodes that have a lower $f(n)$ value, indicating they are closer to the goal or have a better overall evaluation.

This equation is used in informed search algorithms like A* search, where the algorithm aims to minimize the total path cost by

considering both the cost incurred so far and the estimated cost remaining. The algorithm evaluates and selects nodes based on their $f(n)$ values to efficiently find the shortest path to the goal state.

Explored and Expanded Nodes

Here's a simplified explanation of the terms "explored" and "expanded" nodes:

1. **Explored Node:** An explored node refers to a node that has been visited during the search process. When the search algorithm encounters a node, it examines or evaluates it to determine its characteristics, such as its state, cost, or heuristic value. Once a node has been examined, it is marked as explored, indicating that it has been visited and its information has been taken into account.
2. **Expanded Node:** An expanded node refers to a node that has been selected for further exploration or expansion during the search process. When the search algorithm expands a node, it generates or creates its neighboring nodes based on the problem's rules or constraints. These neighboring nodes represent the possible next steps in the search. The expanded node becomes a parent node to its generated neighboring nodes.

In simpler terms, an explored node is a node that has been visited and examined during the search process. It means that the search algorithm has looked at its properties or characteristics. On the other hand, an expanded node is a node that has been selected for further exploration, and its neighboring nodes have been generated. The expanded node becomes the parent of its generated neighboring nodes.

Both explored and expanded nodes are essential in search algorithms as they help keep track of the progress of the search and guide the exploration of the search space.

In the steps of the Best-First Search algorithm, several key concepts are involved. Let's go through them one by one:

1. **Initial State:** This is the starting point of the search. It represents the state from which the search process begins.
2. **Goal State:** This is the desired state that the search aims to reach. It defines the condition that signifies the solution to the problem.
3. **Heuristic Function:** A heuristic function is used to estimate how close a given node is to the goal state. It provides a measure of desirability or potential for each node. The heuristic function guides the search by prioritizing nodes based on their heuristic values.
4. **Priority Queue:** The priority queue is a data structure that stores the nodes during the search process. Nodes are inserted into the priority queue based on their heuristic values. The node with the highest priority (according to the heuristic function) is selected for expansion.
5. **Expansion:** When a node is selected from the priority queue, it is expanded. Expansion involves generating the neighboring nodes or states from the current node. These neighboring nodes represent the possible next steps in the search.
6. **Evaluation:** Each generated neighboring node is evaluated using the heuristic function. The heuristic value is computed to estimate the desirability of the node in reaching the goal state.
7. **Insertion:** The evaluated neighboring nodes are inserted into the priority queue, based on their heuristic values. The priority queue maintains the order of nodes, ensuring that the most promising nodes are explored first.

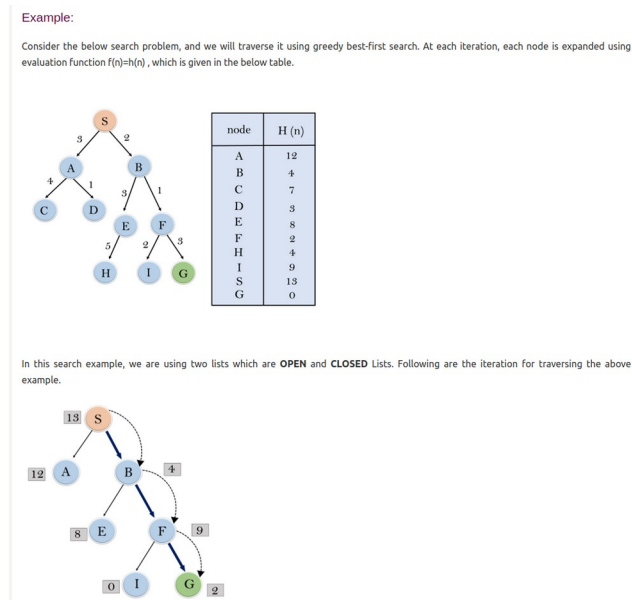
BEST FIRST SEARCH (GREEDY SEARCH)

8. **Termination:** The search process continues until one of the following conditions is met:

- **The goal state is found:** If the selected node from the priority queue matches the goal state, the search terminates, and a solution is found.
- **Open list becomes empty:** If the priority queue (open list) is empty and no solution has been found, it indicates that there is no path to the goal state, and the search terminates without a solution.

Throughout the search process, the Best-First Search algorithm uses the heuristic function to guide the exploration, expanding nodes with higher heuristic values first. The algorithm dynamically adjusts the priority queue based on the evaluation of the neighboring nodes, continually prioritizing the most promising options. This enables the algorithm to efficiently explore the search space and potentially find a solution close to the goal state.

Example :-



Expand the nodes of S and put in the CLOSED list

Initialization: Open [A, B], Closed [S]

Iteration 1: Open [A], Closed [S, B]

Iteration 2: Open [E, F, A], Closed [S, B]
: Open [E, A], Closed [S, B, F]

Iteration 3: Open [I, G, E, A], Closed [S, B, F]
: Open [I, E, A], Closed [S, B, F, G]

Hence the final solution path will be: $S \rightarrow B \rightarrow F \rightarrow G$

Time Complexity: The worst case time complexity of Greedy best first search is $O(b^m)$.

Space Complexity: The worst case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.

Complete: Greedy best-first search is also incomplete, even if the given state space is finite.

Optimal: Greedy best first search algorithm is not optimal.

Properties of Best First Search

Here's a simplified explanation of the characteristics of the Best-First Search algorithm:

1. **Completeness:** Best-First Search is considered complete if repetition (looping) is controlled. Without proper control, it can get stuck in infinite loops and not find a solution. By managing repetition, it can explore the entire search space and find a solution if one exists.
2. **Time Complexity:** The time complexity of Best-First Search is represented as $O(b \wedge m)$, where 'b' is the branching factor (average number of successors per node) and 'm' is the maximum depth of the search space. In simpler terms, the time it takes to execute the algorithm grows exponentially with the branching factor and depth of the problem. However, if a good heuristic is used, it can dramatically improve the algorithm's performance by guiding it towards promising paths.
3. **Space Complexity:** The space complexity of Best-First Search is also represented as $O(b \wedge m)$. It means that the algorithm needs to keep all generated nodes in memory during the search process. The space required increases exponentially with the branching factor and depth of the search space.
4. **Optimality:** Best-First Search does not guarantee finding the optimal solution. Due to its reliance on heuristics, it can overlook certain paths or make suboptimal choices. While it can find a solution

quickly, it may not be the best or optimal solution for the problem.

In simpler terms, Best-First Search explores the search space by prioritizing nodes based on their desirability according to a heuristic function. It can get stuck in loops without proper control, and the time and space required grow exponentially with the branching factor and depth of the problem. While it can find solutions quickly, they may not be the best ones in terms of optimality.

A* ALGORITHM

A* (pronounced "A-star") is a popular informed search algorithm that combines elements of both uniform cost search and best-first search. It is widely used in pathfinding and optimization problems. A* algorithm guarantees finding the optimal solution, provided that certain conditions are met.

Here's a brief overview of how the A* algorithm works:

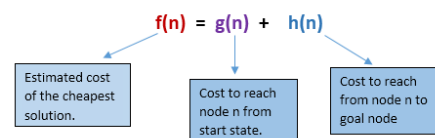
1. Initialize the open list with the initial state and set its cost to zero.
2. Initialize the closed list as an empty set.
3. While the open list is not empty:
 - a. Select the node with the lowest cost from the open list. This is determined by the sum of the cost to reach that node (known as $g(n)$) and the estimated cost from that node to the goal state (known as $h(n)$).
 - b. If the selected node is the goal state, terminate the search and return the solution.
 - c. Move the selected node from the open list to the closed list to mark it as visited.
 - d. Generate the neighboring nodes from the selected node.
 - e. For each neighboring node:
 - Calculate its cost to reach ($g(n)$) by adding the cost from the initial state to the selected node and the cost from the selected node to the neighboring node.
 - If the neighboring node is already in the closed list and the new cost

is higher than the previous cost, skip this node.

- If the neighboring node is not in the open list or the new cost is lower than the previous cost, update the node's cost and set its parent as the selected node.
 - If the neighboring node is not in the open list, calculate its estimated cost to the goal state ($h(n)$).
 - Add the neighboring node to the open list.
4. If the open list becomes empty without finding the goal state, then there is no solution.

The A* algorithm intelligently balances the cost of reaching a node ($g(n)$) and the estimated cost from that node to the goal state ($h(n)$). The heuristic function used in A* should be admissible, meaning it never overestimates the actual cost to reach the goal. If the heuristic is admissible, A* is guaranteed to find the optimal solution.

By considering both the actual cost and the estimated cost, A* can explore the search space efficiently and converge towards the optimal path. It intelligently prioritizes nodes with lower costs, making it more efficient than uninformed search algorithms like Breadth-First Search or Depth-First Search.

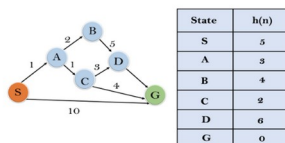


Example :-

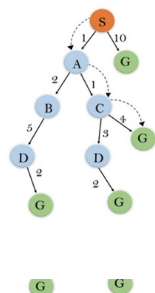
Example:

In this example, we will traverse the given graph using the A* algorithm. The heuristic value of all states is given in the below table so we will calculate the $f(n)$ of each state using the formula $f(n) = g(n) + h(n)$, where $g(n)$ is the cost to reach any node from start state.

Here we will use OPEN and CLOSED list.



Solution:



Initialization: $\{(S, 5)\}$

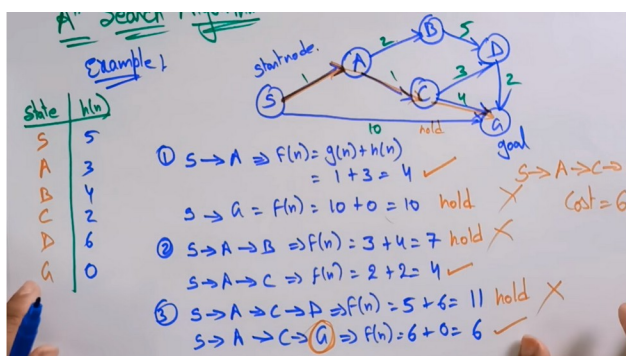
Iteration1: $\{(S \rightarrow A, 4), (S \rightarrow G, 10)\}$

Iteration2: $\{(S \rightarrow A \rightarrow C, 4), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$

Iteration3: $\{(S \rightarrow A \rightarrow C \rightarrow G, 6), (S \rightarrow A \rightarrow C \rightarrow D, 11), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$

Iteration 4 will give the final result, as $S \rightarrow A \rightarrow C \rightarrow G$ it provides the optimal path with cost 6.

Example :-



Properties of A* algorithm

Here's a simplified explanation of the characteristics of the A* algorithm:

- Completeness:** A* search is complete, which means it is guaranteed to find a solution if one exists. It will eventually reach the goal state and terminate the search process.
- Time and Space Complexity:** The time and space complexity of A* search can be exponential. It keeps all generated nodes in memory, and the amount of memory

required grows exponentially with the size of the problem. The actual time and space complexity depend on the specific problem and the quality of the heuristic function used.

- Optimality:** A* search is optimal if the heuristic function used is admissible. An admissible heuristic never overestimates the actual cost to reach the goal state. When an admissible heuristic is used, A* guarantees to find the optimal solution, i.e., the shortest path from the initial state to the goal state.

In simpler terms, A* search is a complete algorithm that will find a solution if there is one. It keeps track of all generated nodes, which can require a significant amount of memory. The time and space complexity can be high, especially in worst-case scenarios. However, the actual performance depends on the problem size and the quality of the heuristic function. If an admissible heuristic is used, A* search is guaranteed to find the optimal solution, providing the shortest path to the goal state.

GAME AS A SEARCH PROBLEM

In artificial intelligence, games can be represented as search problems. Search algorithms are used to navigate through the game's decision tree, exploring possible moves and finding the optimal or best moves to achieve a desired outcome.

- State:** In a game, the state represents the current configuration or situation of the game at a specific point in time. It includes information such as the positions of game pieces, scores, available moves, and any other relevant game-specific data.
- Initial State:** The initial state is the starting point of the game. It represents the initial configuration of the game before any moves have been made.
- Actions:** Actions are the possible moves or decisions that a player can make at any given state of the game. These actions

depend on the rules and mechanics of the specific game being played.

4. **Successor Function:** The successor function defines the result of applying an action to a state. It generates new states by applying legal moves to the current state of the game.
5. **Goal State:** The goal state represents the desired outcome or winning condition of the game. It specifies the condition that indicates a winning position, such as capturing all opponent's pieces, reaching a certain score, or fulfilling a specific objective.
6. **Search Space:** The search space refers to the entire set of possible states that can be reached by applying different actions from the initial state. It represents all the potential paths and configurations the game can take.
7. **Search Algorithm:** A search algorithm is used to navigate through the search space and find the best or optimal moves. Various search algorithms like Minimax, Alpha-Beta Pruning, or Monte Carlo Tree Search (MCTS) can be used depending on the characteristics of the game.
8. **Evaluation Function:** An evaluation function is often used to assign a value to a game state, indicating its desirability or quality. This function is typically used in heuristic-based search algorithms like Minimax with Alpha-Beta Pruning. The evaluation function estimates the strength or advantage of a particular game state for a player.

By representing a game as a search problem, artificial intelligence can utilize search algorithms to explore the potential moves and make informed decisions. The search algorithm analyzes the game states, considers different actions, and evaluates their outcomes to determine the best moves to achieve the desired outcome, such as winning the game or maximizing the player's advantage.

ADVERSARIAL SEARCH

Adversarial search is a type of search problem that arises when multiple agents or players are involved in a game or problem-solving scenario. In this case, each agent aims to find the best solution or make the best decisions while considering the actions and strategies of the other agents who are competing against them.

In traditional search problems, we typically focus on finding a solution that involves a single agent making a sequence of actions. However, in adversarial search, there are multiple agents with conflicting goals who are exploring the same search space.

This type of search is commonly encountered in game playing, where players compete against each other to achieve their individual objectives. Each player needs to think strategically, considering not only their own actions but also the potential actions and moves of their opponents. The decisions made by one player can have an impact on the performance and outcomes of the other players.

Adversarial search involves analyzing and evaluating the game states, considering the possible actions and strategies of both oneself and the opponents. The goal is to find the best moves or decisions that maximize one's own chances of winning or achieving their objectives while taking into account the actions and strategies of the other players.

In summary, adversarial search refers to the problem of finding the best moves or decisions in a game or multi-agent environment, where multiple players with conflicting goals are exploring the same search space and trying to outperform each other. It involves considering the actions and strategies of both oneself and the opponents to make informed and strategic decisions.

A Two Person Zero Sum Game

Here's a simplified explanation of the concepts related to 2-person zero-sum games and perfect information:

1. **2-Person Game:** A 2-person game refers to a game in which two players take turns making moves or decisions. These players could be individuals, teams, or AI agents.
2. **Zero-Sum Game:** A zero-sum game is a type of game where the gains and losses of one player are perfectly balanced with the gains and losses of the other player. In other words, whatever one player gains, the other player loses, and vice versa. The total outcome of the game is zero-sum.
3. **Evaluation Function ($f(n)$):** An evaluation function is a mathematical function used to assess the quality or goodness of a particular position or state in the game. In a zero-sum game, a single evaluation function is used to describe the desirability of a board with respect to both players. Positive values indicate a position favorable to one player, negative values indicate a position favorable to the other player, and values near zero represent neutral positions.
4. **+Infinity and -Infinity:** In the context of the evaluation function, +infinity represents a winning position for one player (Player A), and -infinity represents a winning position for the other player (Player B). These values indicate an overwhelmingly advantageous position for the corresponding player.
5. **Perfect Information:** Perfect information refers to a type of game where both players have access to complete and accurate information about the state of the game. There are no hidden or unknown elements. Both players are fully aware of the current state and past moves made by themselves and the opponent.
6. **Sequential Decision-Making:** In perfect-information games, players take turns

making decisions. When it's a player's turn to act, they have the opportunity to observe the current state of the game before making their move. This allows them to make informed decisions based on the available information.

In simpler terms, a 2-person zero-sum game is a game where two players take turns making moves, and whatever one player gains, the other player loses. An evaluation function is used to assess the quality of positions, with positive values indicating advantage for one player and negative values indicating advantage for the other player. Perfect information means that both players have complete knowledge of the game state. In such games, players make decisions sequentially, observing the current state before choosing their moves.

Example of an adversarial game : Tic-tac-toe

Tic-tac-toe is a classic adversarial game played on a grid of squares, typically a 3x3 grid. The objective of the game is to be the first player to create a straight line of three of their own symbols in a row, column, or diagonal.

The game is played by two players, usually referred to as player X and player O. Player X always makes the first move by placing an X symbol in any of the open squares on the board. Then, player O takes their turn and places an O symbol in any remaining open square. The players continue taking turns, each marking their respective symbol in an empty square.

The game ends in one of three ways :-

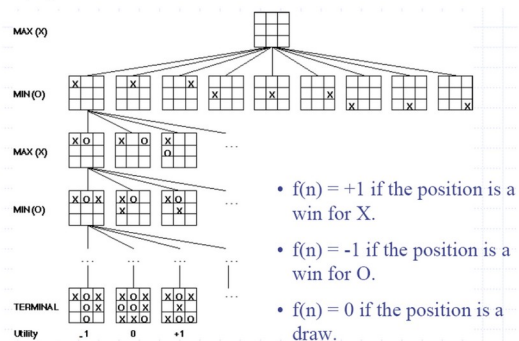
1. If one player successfully creates a line of three of their symbols (X or O) in a row, column, or diagonal, they win the game.
2. If all the squares on the board are filled with symbols, and no player has achieved a winning line, the game is considered a draw or tie.
3. If a winning line is not achieved, but there are still empty squares on the board, the game continues until a winning

line is formed or the board is filled, resulting in a draw.

In summary, Tic-tac-toe is a simple game played on a grid where two players take turns marking X and O symbols on empty squares. The goal is to create a line of three symbols in a row, column, or diagonal. The first player to achieve this wins, and if no player accomplishes this and all squares are filled, the game ends in a draw.

Partial Game Tree for Tic-Tac-Toe

Player 1 (X) moves first



GAME SEARCH TECHNIQUES

MIN-MAX ALGORITHM

ALPHA-BETA PRUNING

